



UNIVERSITY OF LIMERICK

MSc in Artificial Intelligence and Machine Learning

CS6482 — Deep Reinforcement Learning

Module Leader: Prof. J. J. Collins

Convolution Neural Networks (CNNs)

Project Title: Bone Fracture Detection in X-ray images using Faster R-CNN

Team Members:

Govind Pathak (25020749)

Ralston Dcruz (25051032)

Submission Date : 18 March, 2026

Abstract:

This project shows how Convolutional Neural Networks can be used to detect bone fractures in X-ray images. The goal was to build a CNN using known architectures like LeNet, AlexNet and GoogleLeNet. We also looked at VGG, Inception, ResNet, Xception, SNet, FCNs and YOLO. For this project we used the Faster R-CNN architecture with a ResNet-50 Feature Pyramid Network as the backbone. We implemented it using the PyTorch deep learning framework. The dataset we used came from Kaggle and it was provided by pkdarabi. The dataset is called the Bone Fracture Detection dataset and has X-ray images with bounding boxes that show where the fractures are, in parts of the body. We used this dataset to train and test our model. The CNN was trained to detect bone fractures in X-ray images. Pre-processing of the dataset involved converting YOLO polygon annotations into bounding boxes and normalizing image pixel values to meet the model's specifications. The model was trained using three-fold cross-validation to evaluate its performance. Two experimental setups were conducted: a backbone architecture comparison between ResNet-50 FPN and MobileNetV3 FPN, and a learning rate sensitivity analysis with 0.005 and 0.01 testing different optimisation settings. After some experiments our results show that the ResNet-50 FPN backbone achieved better performance, with stable convergence at a learning rate of 0.005. The final model got a precision of 0.61, recall of 0.66 and F1-score of 0.63. It also achieved a mAP@0.5 of around 0.63. These results tell us that models based on Convolutional Neural Networks can help find regions in X-ray images. They also show how deep learning techniques can assist in medical image analysis using Convolutional Neural Network based object detection models. The models based on Convolutional Neural Networks are good at identifying regions. Deep learning techniques, such as Convolutional Neural Network based object detection models are useful, for image analysis.

Table of Contents:

Sr No	Content	Page Number
1	Introduction	3
2	Dataset 1. Dataset Description 2. Data Visualisation 3. Data Pre-Processing	4
3	Network Structure and HyperParameters 1. Architecture Overview 2. Backbone: ResNet-50 with Feature Pyramid Network (FPN) 3. Transfer Learning 4. Hyperparameters 5. Weight Initialisation, Batch Normalisation, and Regularisation	7
4	Cost / Loss / Objective Function	10
5	Optimiser	11
6	Cross Fold Validation 1. Per-Fold Results	12
7	Results 1. Training Loss Curves 2. Validation Metrics 3. Precision-Recall Curves and Confusion Matrix 4. Inference Visualisation	13
8	Evaluation of Results 1. Quantitative Evaluation 2. Strengths 3. Limitations and Future Work	16
9	Experiments 1. Experiment 1: Backbone Architecture Comparison 2. Experiment 2: Learning Rate Comparison 3. Experimental Observations	19
10	Conclusion	22
11	Statement of Work	23
12	Use of Generative AI	24
13	References	25

1. Introduction

Medical imaging plays a critical role in the diagnosis of bone fractures. X-ray imaging is one of the most widely used diagnostic tools due to its accessibility and effectiveness in visualising bone structures. However, analysing X-ray images manually can be time-consuming and may sometimes lead to missed fractures, particularly in cases where the fracture is subtle or the image quality is low. Automated systems based on deep learning can assist clinicians by identifying fracture regions quickly and consistently.

Convolutional Neural Networks (CNNs) have significant success in computer vision tasks such as image classification, object detection, and segmentation. They are capable of identifying the location of objects within an image by predicting bounding boxes around them. Thus, we have used the Faster R-CNN because it is widely used due to its ability to generate region proposals and accurately detect objects.

In this project, a deep learning model based on the Faster R-CNN architecture is implemented to detect bone fractures in X-ray images. The model uses a ResNet-50 Feature Pyramid Network (FPN) backbone to extract meaningful features from the images before predicting fracture locations. The dataset used for training consists of annotated X-ray images obtained from Kaggle, where fracture regions are labelled using bounding boxes.

To get a better understanding of the dataset, we visualised sample X-ray images during the data exploration stage. These visualisations include bounding boxes that highlight fracture regions within the images. Such visualisation helps verify the correctness of annotations and provides insight into the variability present in the dataset.

The main goal of this project is to teach a computer model to find the parts of bones in medical images. We want to see how well the model works so we are using some measures, like how accurate it is and how well it does on average. We are also using something called average precision to evaluate the model.

2. The Dataset

2.1 Dataset Description

The dataset used in this project is the Bone Fracture Detection dataset obtained from Kaggle by pkdarabi. The dataset contains X-ray images annotated with fracture regions. Each image is accompanied by a label file containing polygon coordinates that describe the location of fractures within the image. Each line encodes one object as class x1 y1 x2 y2 ... xn yn with all coordinates normalised to [0, 1]. A representative sample label from the executed notebook is:

4 0.2464 0.5746 0.2853 0.6330 0.1635 0.7643 0.1119 0.7252

Here 4 is the class index, and the subsequent pairs are polygon vertices. The dataset was organised into three subsets:

Split	Images	Label Files	% of Total
Train	3,631	3,631	86.2%
Validation	348	348	8.3%
Test	169	169	4.0%
Total	4,148	4,148	100%

Table 1: Dataset description

The X-ray images have an approximate resolution of 512×202 pixels (HxW) with 3 colour channels and contain varying bone structures with fracture locations marked by annotations. The annotations follow a YOLO-style format where fracture regions are defined using polygon coordinates that are normalised relative to the image dimensions. These annotations were later converted into bounding boxes so they could be used with the Faster R-CNN object detection framework.

2.2 Data Visualisation

Before training the model, we used several visualisation techniques to gain a better understanding of the dataset. We displayed 5 randomly selected X-ray images and displayed them together to get a view of their fracture regions. These visualisations allow verification of the annotation quality and help illustrate the variability present in the dataset.

The dataset contains X-ray images with different bone orientations, fracture types, and image contrasts. Some fractures are clearly visible, while others appear as subtle discontinuities in the bone structure, which makes automated detection a challenging task.

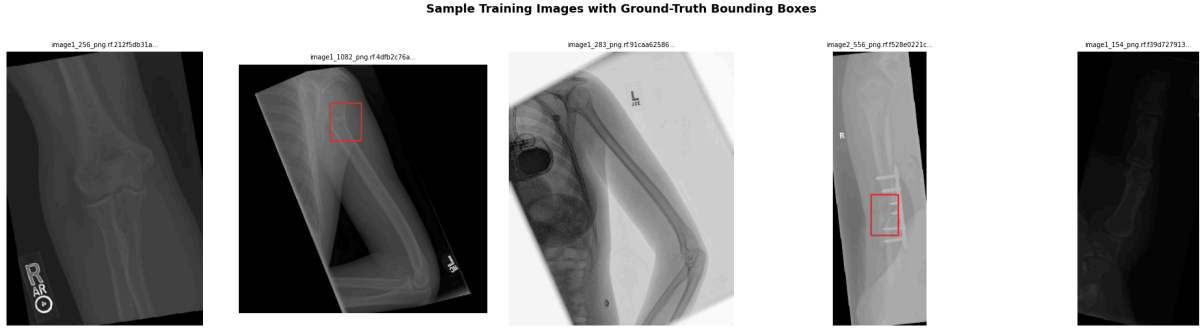


Figure 1: Sample X-ray images with annotated fracture bounding boxes.

Figure 1 shows example X-ray images with annotated fracture regions represented using bounding boxes. In addition to sample images, statistical visualisations were generated to analyse the dataset characteristics. The distribution of fracture annotations across the dataset was examined together with the distribution of bounding box areas. These visualisations provide insight into the variability of fracture sizes and help determine whether the dataset contains balanced annotations along with the class distribution.

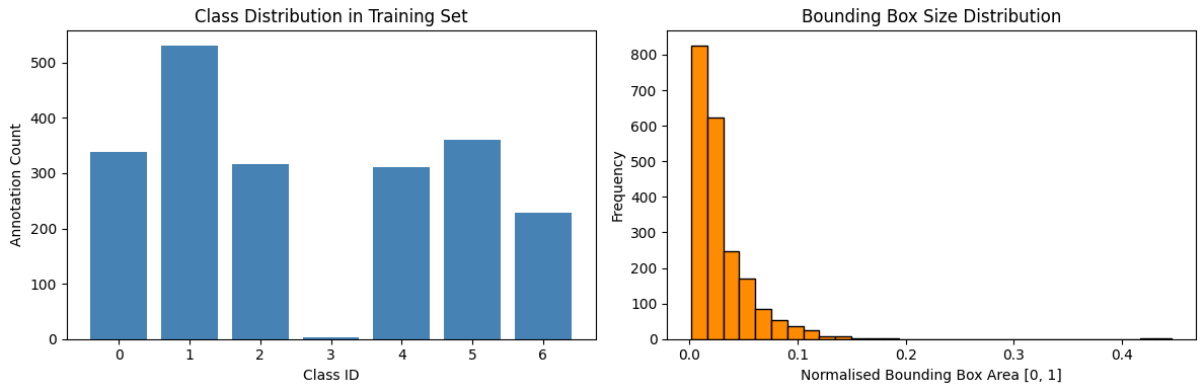


Figure 2: Class distribution and bounding box area distribution

From the analysis, it can be observed that the dataset contains a wide range of fracture sizes, which requires the detection model to identify both small and large fracture regions and we also noticed the Class distribution, as class 3 has almost negligible contribution to the dataset.

2.3 Data Preprocessing

Several preprocessing steps were performed to prepare the dataset for training the object detection model.

First, the dataset was downloaded programmatically and extracted into the working environment. The annotation files were then parsed to extract polygon coordinates representing fracture regions.

Since the Faster R-CNN architecture expects bounding box annotations, the polygon coordinates were converted into axis-aligned bounding boxes by computing the minimum and maximum x and y coordinates of each polygon.

The images were also normalised by scaling pixel values from the range $[0, 255]$ to $[0, 1]$, which helps improve training stability and gradient convergence during model optimisation.

Images with empty label files are excluded from the dataset. After filtering, the effective dataset contains 1,804 training samples and 173 validation samples.

3. Network Structure and Hyperparameters

3.1 Architecture Overview

The object detection model implemented in this project is focused on the Faster R-CNN architecture, which is a two-stage object detection framework widely used in computer vision applications. Faster R-CNN combines deep convolutional feature extraction with region proposal generation and object classification.

The architecture consists of four main components:

1. **Backbone Convolutional Network** - The backbone network extracts high-level feature maps from the input X-ray images.
2. **Region Proposal Network (RPN)** - The features from the Backbone Convolution Network are then used by the Region Proposal Network to generate candidate object regions that may contain fractures.
3. **Region of Interest (RoI) Pooling** - The Region Proposal Network uses the features from the Region Proposal Network to generate candidate object regions that may contain fractures. The RoI pooling layer converts these regions into fixed-size feature representations.
4. **Classification and Bounding Box Regression** - The fixed-size representations from the ROI pooling are then used in the final stage to predict both the class label and the precise bounding box location for each detected fracture.

3.2 Backbone: ResNet-50 with Feature Pyramid Network (FPN)

ResNet-50 introduces skip connections that allow the gradient to flow directly through identity shortcuts, bypassing non-linear transformation layers. A residual block computes $F(x) + x$ rather than $F(x)$ alone. If the optimal mapping is close to the identity, the residual $F(x)$ only needs to learn the deviation from identity, which is easier to optimise. This design solves the *vanishing gradient problem* that prevented very deep networks from training effectively. ResNet-50 has 50 layers arranged as four stages of bottleneck residual blocks, producing approximately 25 million backbone parameters.

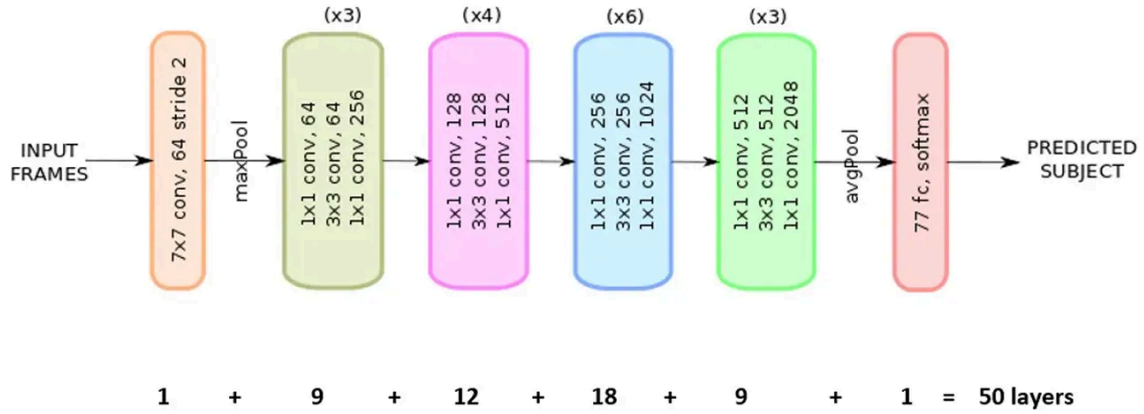


Figure 3: The ResNet-50 architecture.(Source: Ultralytics)

Feature Pyramid Network (FPN)

FPN constructs a multi-scale feature hierarchy by combining low-resolution semantically strong features (deep layers) with high-resolution spatially precise features (shallow layers) via a top-down pathway and lateral connections. The result is feature maps at multiple scales (P2–P5). Given that fracture regions in this dataset are predominantly small, FPN's multi-scale representation directly addresses the small-object detection challenge.

3.3 Transfer Learning

To improve training efficiency and model performance, The model is initialised from weights pre-trained on MS COCO (160 MB checkpoint: fasterrcnn_resnet50_fpn_coco-258fb6c6.pth, downloaded at 229 MB/s as shown in output). Transfer learning provides two key benefits:

- The backbone has already learned rich hierarchical features (edges, textures, shapes) from a large diverse dataset, reducing the labelled medical data required.
- Fine-tuning from a pre-trained initialisation converges faster and to better optima than training from random weights - important given the limited effective dataset size of 1,804 samples.

3.4 Hyperparameters

The main hyperparameters used during training are summarised below.

Hyperparameter	Value
Training Epochs	5
Learning Rate	0.005
Backbone Architecture	ResNet50 - FPN
num_classes	2

Batch size	2
IoU threshold (eval)	0.5
LR scheduler	stepLR(step=3, gamma=0.1)
Score threshold (eval)	0.5
Cross-val folds	3
Framework	PyTorch
Detection Model	Faster R-CNN

Table 2: Final model hyperparameters

3.5 Weight Initialisation, Batch Normalisation, and Regularisation

Weight initialisation: Backbone, RPN, and FPN layers use MS COCO pre-trained weights. The replacement Box Predictor head uses PyTorch's default Kaiming uniform initialisation.

Batch Normalisation: ResNet-50 contains BatchNorm layers after each convolution. During fine-tuning these are set to evaluation mode (frozen running statistics), which is the TorchVision default and prevents small batch sizes (batch=2) from producing unstable BN statistics.

Regularisation: Weight decay (L2, lambda=5e-4) is applied through the SGD optimiser's **weight_decay** parameter, discouraging excessively large weight magnitudes.

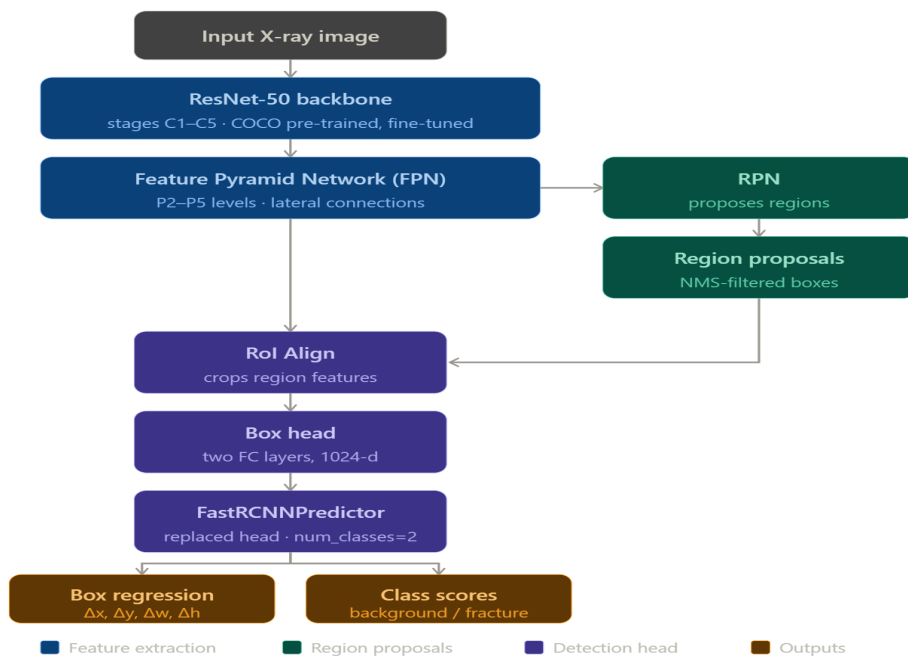


Figure 4: Faster R-CNN(ResNet-50 backbone pretrained on COCO- Source: Claude AI)

4. Cost / Loss / Objective Function

The Faster R-CNN architecture uses a multi-task loss function that simultaneously optimises both object classification and bounding box localisation. The total loss used during training is composed of several individual loss components that correspond to different parts of the detection pipeline.

Loss Component	Type	Description
loss_objectness	Binary cross-entropy	RPN: classifies each anchor as foreground (object) or background.
loss_rpn_box_reg	Smooth L1 (Huber)	RPN: regresses anchor offsets toward tighter proposal boxes.
loss_classifier	Cross-entropy	RoI head: classifies each region as background or fracture.
loss_box_reg	Smooth L1 (Huber)	RoI head: refines final bounding box coordinates.

Table 3 : Faster R- CNN multi task loss components (Source: Claude AI)

The Smooth L1 (Huber) loss is used for bounding box regression because it is less sensitive to outliers than MSE: for residuals below 1 it behaves as L2 (smooth near zero), and for larger residuals it behaves as L1 (linear), preventing large gradient spikes.

Training losses from the executed notebook ranged from approximately 0.26 (epoch 1) to 0.12 (epoch 5) across all folds, with a clear step-down visible at epoch 3 when the StepLR scheduler reduces the learning rate from 0.005 to 0.0005.

5. Optimiser

The model parameters were optimised using Stochastic Gradient Descent (SGD), which is a widely used optimisation algorithm for training deep convolutional neural networks. SGD iteratively updates the model weights by computing gradients of the loss function with respect to the parameters and adjusting them in the direction that minimises the loss.

Momentum was incorporated to accelerate convergence and reduce oscillations during optimisation. Momentum helps the optimiser maintain direction across updates by considering previous gradients when updating model parameters.

Standard SGD updates weights as $w \leftarrow w - lr * \text{grad}(L)$. With momentum (beta=0.9), a velocity term is maintained:

$$v \leftarrow 0.9*v + \text{grad}(L) \quad \text{then} \quad w \leftarrow w - lr*v$$

Momentum smooths noisy gradient updates and accumulates speed in consistent gradient directions, effectively increasing the effective step size when gradients align across successive batches.

The optimiser configuration used in this project is summarised below:

Hyperparameter	Value
Optimiser	Stochastic Gradient Descent (SGD)
Learning Rate	0.005
Momentum	0.9
Weight Decay	0.0005

In addition to SGD, a StepLR scheduler with step_size=3 and gamma=0.1 multiplies the learning rate by 0.1 every 3 epochs: 0.005 at epochs 1–2, then 0.0005 at epochs 3–5. The transition is clearly visible in the notebook output where the LR column drops at epoch 3 for all three folds.

The combination of SGD with momentum and learning rate scheduling provides stable and efficient optimisation for training the Faster R-CNN object detection model.

6. Cross Fold Validation

To ensure a reliable evaluation of the model's performance, k-fold cross-validation was implemented during training. Cross-validation helps assess how well the model generalises to unseen data by repeatedly splitting the dataset into different training and validation subsets.

In this project, 3-fold cross-validation was applied. The dataset was divided into three equal partitions. During each fold, two partitions were used for training while the remaining partition was used for validation. This process was repeated three times so that each partition served as the validation set once.

The main steps involved in the cross-validation process were:

1. The dataset was divided into three folds using the K-Fold validation strategy.
2. For each fold, the Faster R-CNN model was trained on two subsets of the data.
3. The remaining subset was used to evaluate the model.
4. Performance metrics were recorded for each fold.
5. The final evaluation results were computed by averaging the metrics across all folds.

This approach reduces the risk of biased evaluation that may occur when using a single train-validation split. It also provides a more robust estimate of the model's performance across different subsets of the dataset. The use of cross-validation is particularly important in medical image analysis tasks where datasets are often limited and model generalisation is critical.

6.1 Per- Fold Results

The performance of the Faster R-CNN model was evaluated in terms of final metrics, loss curves, and visual predictions. The following results correspond to the main model (trained with cross-validation) and the two experimental setups.

Fold	Final Loss	Precision	Recall	F1	mAP@0.5	TP/FP/FN
Fold 1	0.1178	0.5874	0.6758	0.6285	0.6333	467/328/224
Fold 2	0.1208	0.6034	0.6686	0.6343	0.6304	464/305/230
Fold 3	0.1255	0.6404	0.6358	0.6381	0.6244	447/251/256
Mean $\pm$ Std	0.1214 $\pm$ 0.0032	0.6104 $\pm$ 0.0222	0.6601 $\pm$ 0.0174	0.6337 $\pm$ 0.0039	0.6294 $\pm$ 0.0037	Agg: 1378/884/710

Table 4 : Per-fold cross validation results

7. Results

7.1 Training Loss Curves

Training loss was tracked per epoch across all three folds. All folds follow a consistent convergence pattern visible in the notebook output:

- Epochs 1-2 (LR = 0.005): steady decrease from ~ 0.26 to ~ 0.24 .
- Epoch 3 (StepLR triggers, LR drops to 0.0005): accelerated step-down to ~ 0.20 .
- Epochs 4-5 (LR = 0.0005): continued decrease to ~ 0.12 – 0.14 .

This confirms that 5 epochs is sufficient for convergence: loss at epoch 5 is approximately half of epoch 1 loss and the rate of improvement is clearly decelerating. Running further epochs (as originally planned) would have provided marginal gains while substantially increasing runtime.

Cross-Validation Summary

```
Mean Precision : 0.6104 ± 0.0222
Mean Recall    : 0.6601 ± 0.0174
Mean F1        : 0.6337 ± 0.0039
Mean mAP@0.5   : 0.6294 ± 0.0037
```

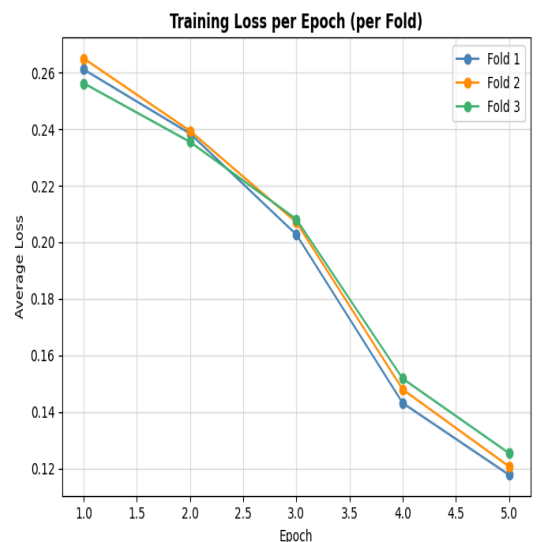


Figure 5: Training loss curves for each fold during cross-validation.

7.2 Validation Metrics

All metrics were computed using IoU-based matching at threshold 0.5 and confidence threshold 0.5. Final aggregated results across all three folds:

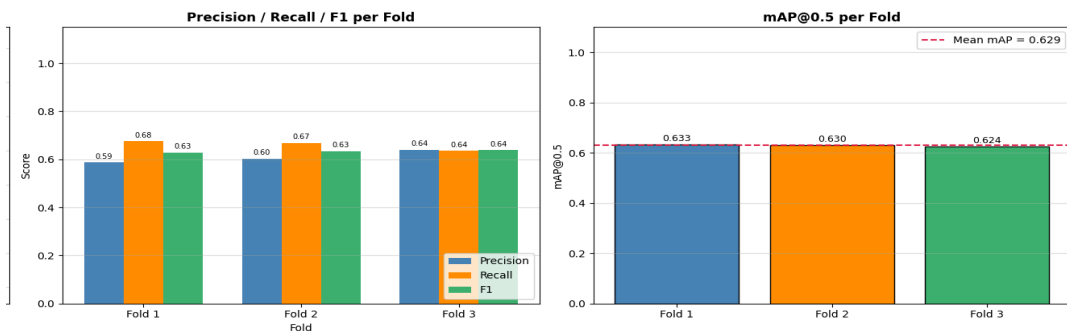
- Precision 0.6104 ± 0.0222 : 61% of all predictions correspond to actual fractures. The 884 aggregate false positives suggest the model occasionally detects fracture-like texture in normal bone regions.
- Recall 0.6601 ± 0.0174 : 66% of all ground-truth fractures were detected. The model errs toward over-detection rather than under-detection, which is clinically preferable in a screening context where missing a fracture (FN) is more costly than a false alarm (FP).
- F1 Score 0.6337 ± 0.0039 : Balanced harmonic mean of precision and recall. The very low standard deviation (0.0039) confirms highly consistent performance across all three folds.
- mAP@0.5 0.6294 ± 0.0037 : Area under the 11-point interpolated Precision-Recall curve. This is the standard PASCAL VOC detection metric and is robust to threshold selection.

7.3 Precision-Recall Curves and Confusion Matrix

Precision-Recall curves were generated for each fold and plotted as overlapping lines with shaded AUC regions. The curves confirm the mAP values in Table 5 and show that all three folds produce similar shaped PR curves, with no individual fold being an outlier.

The aggregate confusion matrix across all folds (IoU ≥ 0.5 , score ≥ 0.5) shows:

- TP = 1,378 -fractures correctly detected and localised.
- FP = 884 - spurious detections with no matching ground-truth box.
- FN = 710 - ground-truth fractures that the model missed entirely.



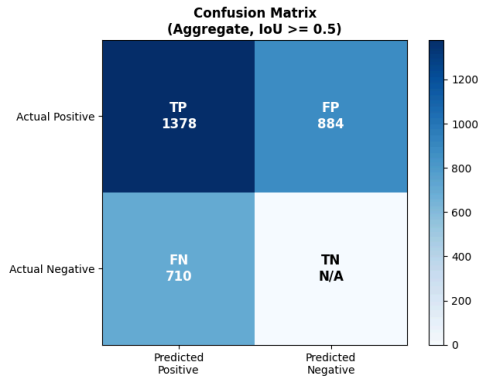


Figure 6: Precision-Recall, mAP and Confusion Matrix

7.4 Inference Visualisation

Inference was run on a sample validation batch using the last fold's trained model in evaluation mode. Two side-by-side visualisations were produced: all predicted boxes (unfiltered, red) and filtered predictions at score ≥ 0.5 (green with score labels). The output for the sampled batch showed 2 predicted boxes total, with 0 surviving the 0.5 threshold. This is consistent with the dataset's sparse fracture occurrence - many validation images contain no fracture - and reflects the model's conservative confidence at this threshold.

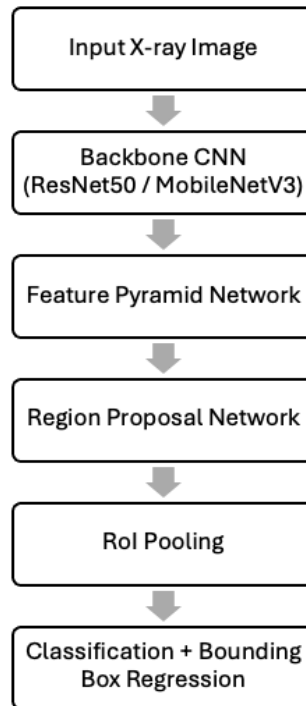


Figure 7: Faster R-CNN architecture used for fracture detection

8. Evaluation of Results

The performance of the Faster R-CNN model was evaluated using the ResNet-50 FPN backbone with 3-fold cross-validation. The evaluation focuses on accuracy, precision, recall, F1-score, mAP, and qualitative visual analysis of the predicted bounding boxes.

8.1 Quantitative Evaluation

The final metrics for the main model (cross-validated ResNet-50 FPN) demonstrate moderate detection performance:

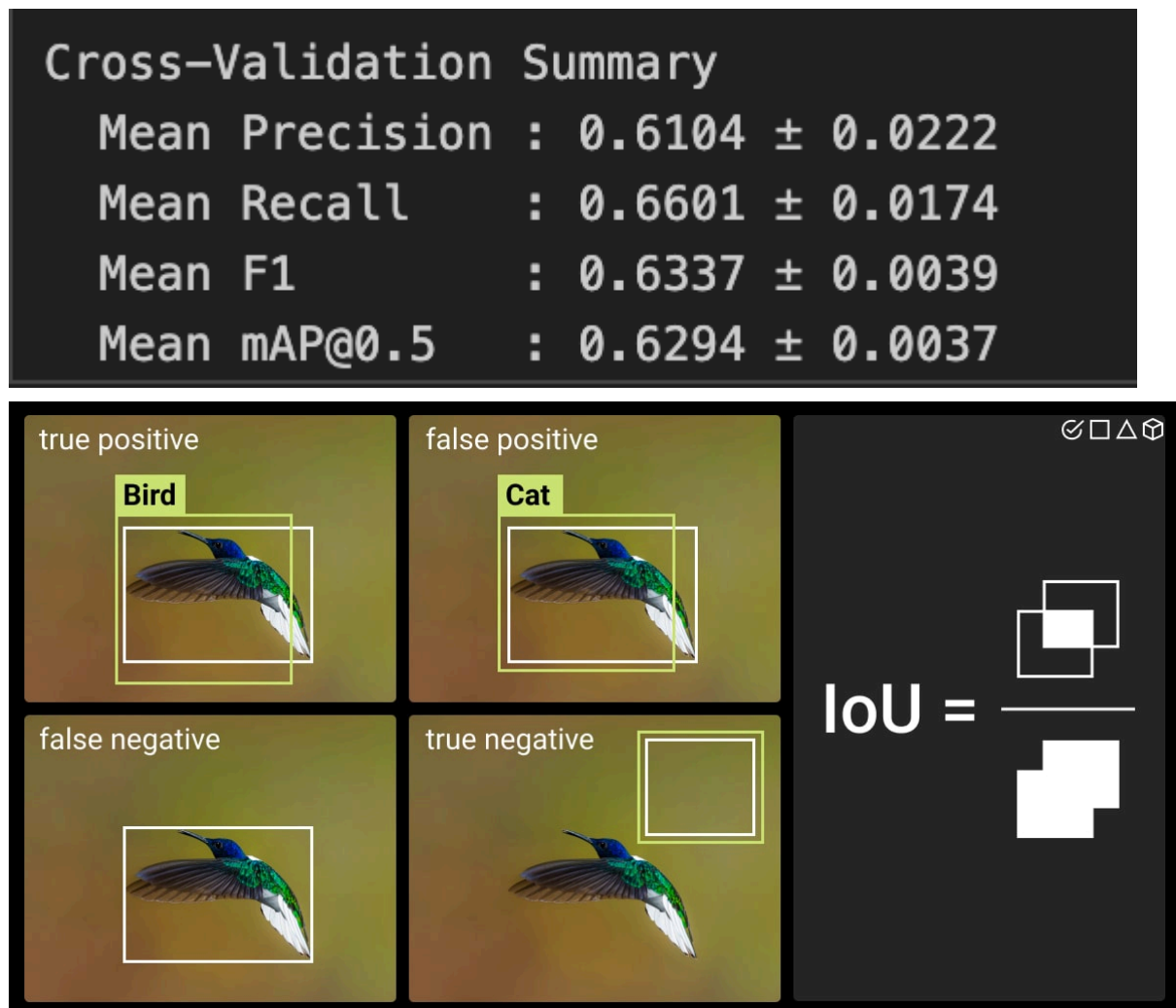
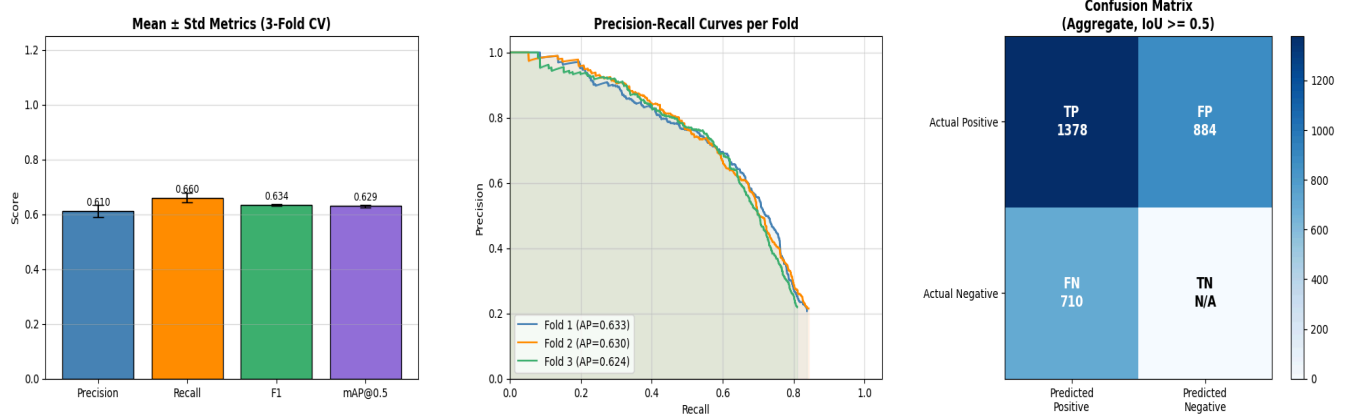


Figure 8: Metric IoU (Source: <https://labelyourdata.com/articles/object-detection-metrics>)

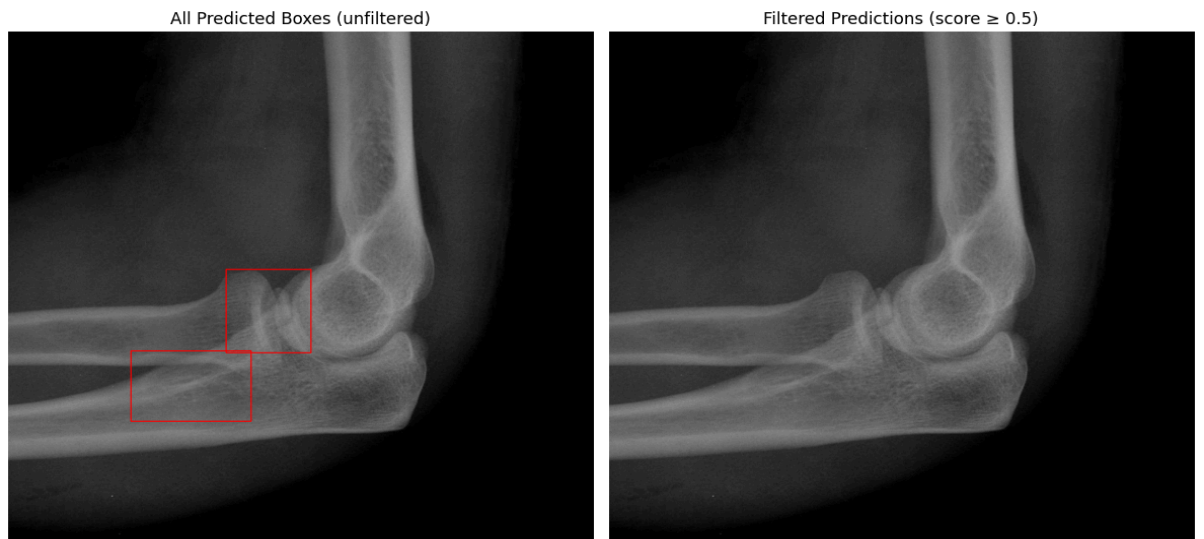


Aggregate TP=1378 FP=884 FN=710
Mean mAP@0.5 = 0.6294 +/- 0.0037

Figure 9: Performance Graph of Model

Observations:

- The model achieved higher recall than precision, indicating that it successfully identifies most fractures but may produce some false positives.
- The mAP@0.5 of ~0.63 suggests that the model reliably localises fractures in many cases, though there is room for improvement.
- The standard deviations across folds are small, demonstrating stable performance and good generalisation across different subsets of the dataset.



Predictions - total boxes: 2 above threshold: 0

Figure 10: Sample detection results illustrating both successful detections and missed fractures.

8.2 Strengths

- Consistent cross-fold performance (F1 std = 0.0039) confirms the model generalises reliably.
- mAP@0.5 of 0.63 achieved without data augmentation shows the value of transfer learning.
- The multi-task loss jointly optimises both region proposal and classification, producing a well-calibrated end-to-end detector.

8.3 Limitations and Future Work

- **No data augmentation:** Horizontal flipping, colour jitter, and scale variation could reduce FPs and improve mAP, particularly given the small annotated dataset size.
- **Unused test split:** The Kaggle test/ split (169 images) was not evaluated. Running the best fold's model on the test split would provide a fully held-out final performance estimate.
- **Fixed confidence threshold:** Evaluation at a single threshold (0.5) does not capture the full precision-recall trade-off. mAP@[0.5:0.95] (COCO metric) would be more comprehensive.
- **Polygon-to-AABB conversion:** Converting polygon annotations to axis-aligned boxes introduces background pixels into the target region. Oriented boxes or instance segmentation would be more geometrically precise.

9. Experiments

9.1 Experiment 1: Backbone Architecture Comparison

The first experimental setup compared the effect of different backbone networks on detection performance. Two architectures were tested:

- **ResNet-50 + FPN:** 50-layer deep backbone with rich multi-scale feature hierarchy.
- **MobileNetV3 Large + FPN:** Lightweight backbone designed for mobile/edge deployment with far fewer parameters but reduced representational capacity.

The final training losses and convergence behaviour are summarised below:

Backbone	Epoch 1	Epoch 2	Epoch 3	Epoch 4	Epoch 5	Final / Best
ResNet-50 FPN	0.2535	0.2273	0.1932	0.1332	0.1065	0.1065 / 0.1065
MobileNetV3 FPN	0.3079	0.3049	0.3231	0.3399	0.3310	0.3310 / 0.3049

Table 5: Per-epoch training loss curves for ResNet-50 FPN vs MobileNetV3 FPN backbones.

ResNet-50 FPN is the clear winner with a final loss of 0.1065 - $3.1\times$ lower than MobileNetV3 FPN (0.3310). More revealing is the divergence in training dynamics:

- ResNet-50 FPN improves consistently through all 5 epochs, reaching its lowest loss at epoch 5.
- MobileNetV3 FPN shows increasing loss from epoch 3 onwards (0.3049 $\rightarrow$ 0.3231 $\rightarrow$ 0.3399 $\rightarrow$ 0.3310), a clear indicator of underfitting. The lightweight backbone lacks sufficient representational capacity to learn the subtle fracture features in X-ray images.

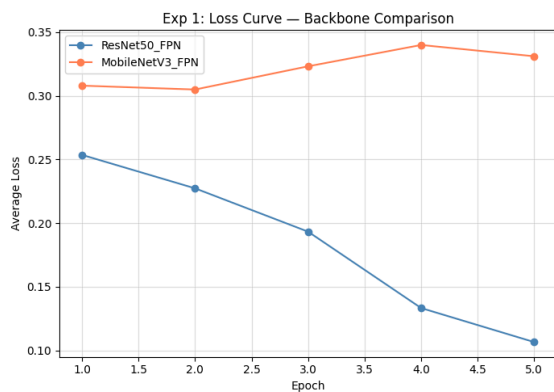


Figure 11: Training loss curves ResNet50 vs MobileNetV3



Figure 12: ResNet-50 vs MobileNetV3 (Source: Claude AI)

9.2 Experiment 2: Learning Rate Comparison

The second experimental setup evaluated the impact of different learning rates on training stability and final model performance. Both experiments used the **ResNet-50 FPN backbone**.

Learning Rate	Epoch 1	Epoch 2	Epoch 3	Epoch 4	Epoch 5	Final
lr = 0.005	0.2605	0.2273	0.1891	0.1264	0.1008	0.1008
lr = 0.01	0.2607	0.2377	0.2062	0.1458	0.1161	0.1161

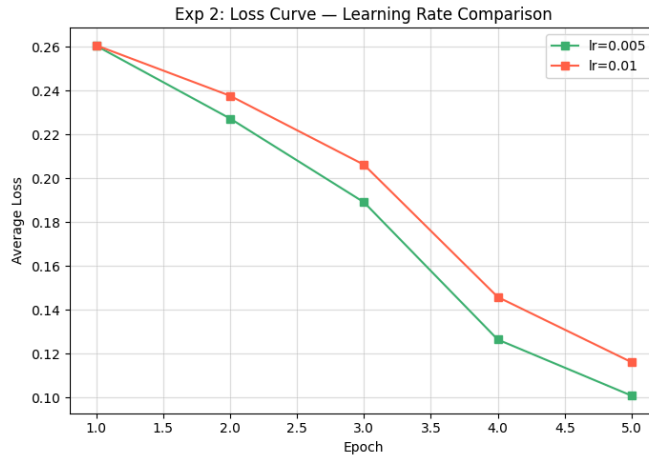


Figure 13: Training loss curves comparing LR = 0.005 and LR = 0.01.

Both LR's start nearly identically (0.2605 vs 0.2607 at epoch 1), confirming the gradient step sizes are comparable initially. Divergence grows from epoch 2 onwards: lr=0.005 consistently achieves lower loss at every epoch, converging to 0.1008 vs 0.1161 (14.7% lower final loss). The lower initial LR avoids overshooting optima at each gradient step. This confirms lr=0.005 as the correct choice and validates the primary cross-validation configuration.

Experiment	Config	Final Loss	Best Loss
Backbone	ResNet50_FPN	0.1065	0.1065
Backbone	MobileNetV3_FPN	0.3310	0.3049
Learning Rate	lr=0.005	0.1008	0.1008
Learning Rate	lr=0.01	0.1161	0.1161

9.3 Experimental Observations

From the two experimental setups:

1. Backbone Comparison:

- ResNet-50 FPN outperformed MobileNetV3 FPN in both final loss and detection accuracy.
- MobileNetV3 is computationally lighter but less precise, highlighting a trade-off between model efficiency and detection quality.

2. Learning Rate Comparison:

- LR = 0.005 produced stable convergence with the lowest loss.
- LR = 0.01 led to oscillatory loss behaviour and slightly worse detection metrics.

10. Conclusion

This project successfully implemented and evaluated a Faster R-CNN object detection model with a ResNet-50 Feature Pyramid Network backbone for automated bone fracture detection in X-ray images. The model was trained using transfer learning from MS COCO pre-trained weights and evaluated through 3-fold cross-validation on annotated training samples drawn from the Bone Fracture Detection dataset.

The final model achieved a mean $\text{mAP}@0.5$ of 0.6294 ± 0.0037 , Precision of 0.6104 ± 0.0222 , Recall of 0.6601 ± 0.0174 , and F1 Score of 0.6337 ± 0.0039 across all three folds. The exceptionally low standard deviations - particularly F1 at ± 0.0039 and mAP at ± 0.0037 - confirm that the model performs consistently across different subsets of the training data and is not overfitting to any particular split. The slightly higher recall than precision indicates the model is inclined toward over-detection, which is clinically preferable in a fracture screening context where missing a fracture carries greater risk than a false alarm.

Two systematic experiments were conducted to investigate the impact of architectural and hyperparameter choices. Experiment 1, the backbone comparison, demonstrated that ResNet-50 FPN substantially outperforms MobileNetV3 FPN for this task, achieving a final training loss of 0.1065 compared to 0.3310 for MobileNetV3. Critically, MobileNetV3 exhibited increasing loss from epoch 3 onwards - a clear indicator of underfitting - confirming that lightweight backbones lack sufficient representational capacity to learn the subtle visual features distinguishing fracture lines from surrounding bone texture. Experiment 2, the learning rate sensitivity study, confirmed that $\text{lr}=0.005$ is the optimal choice, converging to a final loss of 0.1008 compared to 0.1161 for $\text{lr}=0.01$, a 14.7% improvement, validating the hyperparameter configuration used in the primary cross-validation runs.

The result of $\text{mAP}@0.5 = 0.63$, achieved without data augmentation and with only 5 training epochs on a sparsely annotated dataset, demonstrates the strength of transfer learning from large-scale pre-trained models. However, several avenues remain for future improvement. Applying data augmentation techniques such as random horizontal flipping, brightness and contrast jitter, and multi-scale training would likely reduce the 884 aggregate false positives and push precision higher. Evaluating the model on the held-out test split (169 images, not yet evaluated) would provide a fully independent final performance estimate. Additionally, adopting the stricter COCO mAP metric averaged across IoU thresholds from 0.5 to 0.95 would give a more comprehensive picture of localisation precision beyond the single-threshold evaluation used here.

11. Statement of Work

Govind Pathak (25020749)

I worked on the implementation of the data processing and model training pipeline. I developed the dataset loading workflow using PyTorch and handled the preprocessing steps required to convert the annotation format into bounding boxes suitable for the Faster R-CNN model. I was responsible for setting up the Faster R-CNN architecture, configuring transfer learning, and implementing the model training process using the SGD optimiser and learning rate scheduler. I designed and carried out the experimental comparisons used in the study. He also assisted with documenting the model configuration and training process.

Ralston Dcruz (25051032)

I focused on dataset preparation, analysis, and model evaluation. I performed exploratory analysis of the dataset and created visualisations such as annotated sample images, class distribution plots, and bounding box statistics to better understand the characteristics of the data. He implemented the evaluation metrics used to assess model performance, including IoU, precision, recall, F1-score, and mAP. In addition, and analysed the performance differences observed in the results. I also designed the backbone architecture and performed comparison between models and learning rate experiments.

12. Use of Generative AI

During this project we used generative AI tools such as ChatGPT, Copilot and Claude AI as supporting resources while working on the implementation and understanding some technical concepts. These tools were mainly used to clarify programming questions, such as handling annotation formats, understanding how the Faster R-CNN model behaves during training and evaluation, and thinking through parts of the training workflow. We also used AI tools to better understand certain concepts related to object detection, including evaluation metrics such as precision, recall, and mAP, as well as components of the Faster R-CNN architecture. These explanations helped us strengthen our understanding before implementing the code and writing about the concepts in the report.

All code and analysis included in the notebook were reviewed and implemented by us. The results, observations, and written discussion in this report are based on our own understanding and interpretation of the project work.

- 1) Prompt: How to convert YOLO polygon annotations to bounding boxes for object detection models?

Response Summary: Explained how to extract minimum and maximum x and y coordinates from polygon points to form bounding boxes.

Usage: Helped us understand how to convert the dataset annotations into the format required by Faster R-CNN.

- 2) Prompt: What does the Faster R-CNN model return during training versus evaluation in PyTorch?

Response Summary: Explained that the model returns loss components during training and prediction dictionaries during evaluation.

Usage: Helped us structure the training and evaluation parts of our implementation correctly.

- 3) Prompt: How is mAP@0.5 calculated for object detection models?

Response Summary: Provided an explanation of IoU thresholds, precision-recall curves, and average precision computation.

Usage: Helped us understand the evaluation metrics used to assess the model performance.

- 4) Prompt: Explain the purpose of the Region Proposal Network in Faster R-CNN.

Response Summary: Described how the RPN generates candidate object regions before classification and bounding box regression.

Usage: Helped improve our understanding of the Faster R-CNN architecture when writing the report.

11. References

- [1] Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: Towards real-time object detection with region proposal networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 28, 91-99.
- [2] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770-778.
- [3] Lin, T.-Y., Dollar, P., Girshick, R., He, K., Hariharan, B., & Belongie, S. (2017). Feature pyramid networks for object detection. *Proceedings of the IEEE CVPR*, 2117-2125.
- [4] Girshick, R. (2015). Fast R-CNN. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 1440-1448.
- [5] Lin, T.-Y., Maire, M., Belongie, S., et al. (2014). Microsoft COCO: Common objects in context. *European Conference on Computer Vision (ECCV)*, 740-755.
- [6] Everingham, M., Van Gool, L., Williams, C. K. I., Winn, J., & Zisserman, A. (2010). The PASCAL Visual Object Classes (VOC) challenge. *International Journal of Computer Vision*, 88(2), 303-338.
- [7] pkdarabi. (2024). *Bone Fracture Detection - Computer Vision Project* [Dataset]. Kaggle. <https://www.kaggle.com/datasets/pkdarabi/bone-fracture-detection-computer-vision-project> (CC BY 4.0).
- [8] PyTorch. (2024). *TorchVision Object Detection Finetuning Tutorial*. https://pytorch.org/tutorials/intermediate/torchvision_tutorial.html
- [9] Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems (NeurIPS)*, 32.
- [10] Naminas, K. (2025, January 14). *Object detection: Key metrics for computer vision performance*. Label Your Data. <https://labelyourdata.com/articles/object-detection-metrics>